Feature 1 Review Transcript

Nov 12, 2026 · @Rahul

A 35-minute walkthrough of how Feature 1 (Stories 1.1 to 1.5) is delivered the way the manager asked: no scripts in the environment, and the RAG approach with IBM Bob or GitLab Duo agents. It explains the steps; it doesn't assume any specs or other outputs exist yet.

Before the session

You only need the package open; nothing has to be run beforehand.

Open in a tab	To show
GitLab: the `iln-specs` repository	`AGENTS.md`, the `agents/` folder, `agents/templates/`, `agents/checklists.md`
Bob, with `iln-specs.code-workspace` open	The mode list showing the six *iln* modes
This transcript	Fill in `<FEATURE-KEY>` and `<JIRA-KEY>` first

Invite: your manager, the lead, the Product Owner, the Mainframe SME, and if possible the GitLab admin and DevOps. Some of the asks at the end are theirs.

1. Opening (2 minutes)

Say: "Thank you for joining. Today I'll explain how we will deliver Feature 1, Stories 1.1 to 1.5: building a full specification of the iln application from its code.

We're following the direction we were given: no scripting in the environment, and a RAG-based approach using the agents we already have, IBM Bob or GitLab Duo. So there is nothing to install. What we provide is a set of instructions, templates and checklists that Bob and Duo follow. The agents read the iln code, and people review and approve what they write.

I'll explain the approach, the building blocks, then the seven steps that take us through the five stories. I'll finish with the limits and what I need from you."

Item	Minutes
The approach in one picture	3

Item	Minutes
The building blocks	3
Steps 1 to 5: one per story	20
Steps 6 and 7: review, approval and progress	3
Limits, asks and next steps	3
Questions	Remaining time

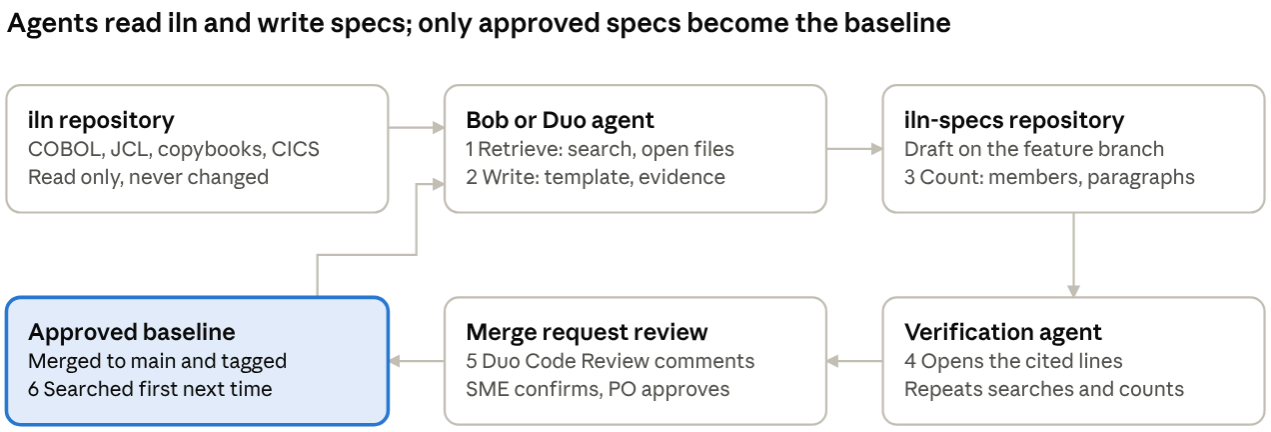
2. The approach in one picture (3 minutes)

Bob or Duo finds the code, writes the specification from a template with evidence for every statement, and people approve it in a merge request.

Say: "RAG means retrieve, augment, generate. Instead of us building a search index with scripts, Bob and Duo use their own search. This is how one piece of the specification gets made, in six steps:

1. **Retrieve.** The agent searches the iln code: for example `PGM=` to find the jobs that run a program, or `COPY` to find who uses a copybook. Duo can also search by meaning when semantic code search is turned on.
2. **Write.** It opens the files it found and fills a fixed template in business words, citing the file and line numbers for every statement, with a confidence level.
3. **Count.** It proves nothing was skipped: every member in a folder, and every paragraph in a program, is accounted for.
4. **Verify.** A separate Verification agent opens the cited lines and repeats the searches and counts.
5. **Review.** Duo Code Review comments on the merge request, the SME confirms it matches the code, and the Product Owner approves the behaviour.
6. **Reuse.** Once approved, the specs are the first thing Bob and Duo search next time. The more we specify, the better every later answer gets.

The iln code is only read, never changed. The specifications live in their own repository, `iln-specs`."



how a spec is made · 6 steps

The agent reads `iln` but never changes it. A spec reaches `main` only after verification and the SME's and Product Owner's approval, and from then on it is the first thing the agents search.

3. The building blocks (3 minutes)

Everything is Markdown and settings files in the `iln-specs` repository; Bob and Duo read them.

Show: `AGENTS.md`, then open one agent file to show its four parts.

Say: "There are six agents. Each one is a single file with four parts: who you are, what to do, what not to do, and what to write. They all follow the same 14 rules in `AGENTS.md`, which Bob and Duo read automatically."

Agent	Job	In Bob
Specification	Inventories the code, maps the connections, writes the specs	Mode *iln Specification*; `/inventory-folder`, `/specify-program`, `/map-connections`
Jira	Runs a story, starting with impact analysis	Mode *iln Jira*; `/impact-analysis`
Verification	Checks every other agent's work	Mode *iln Verification*; `/verify-spec`
Architect	Records the design patterns in the code	Mode *iln Architect*
Testing	Writes tests from the specs	Mode *iln Testing*

Agent	Job	In Bob
Coding	Changes code for a story, after approval	Mode *iln Coding*

Say: "Four more pieces make it work:

- **Templates** in `agents/templates/` : the fixed shape of every output. They are our specification standard.
- **Checklists** in `agents/checklists.md` : the quality checks, used by the agents, Duo Code Review and the reviewers.
- **Bob modes**: each agent can edit only its own folders, and none can run commands.
- **Duo settings**: chat rules, plus review instructions so Duo checks every merge request against our checklist."

4. Step 1: set up the environment (Story 1.1, 4 minutes)

Definition of Done: a specification can be stored, versioned, retrieved by an agent, processed, and traced through an agent run.

Say: "The first step is the controlled environment. Every control is either a GitLab setting, a Bob or Duo setting, or a reviewed file in the repository. No scripts."

How it runs:

1. **Repository:** `iln-specs` in GitLab, with protected `main` and code-owner approval. The SME and Product Owner must approve every spec change.
2. **Bob:** open `iln-specs.code-workspace` . Bob loads the rules, six modes and skills from `iln-specs` , and reads the code from `iln` .
3. **Duo:** the admin turns on Duo, the context exclusions, Duo Code Review and, if available, semantic code search on `iln` .
4. **Secrets:** Duo's context exclusions and `.bobignore` keep restricted files away from the model; secret push protection blocks keys and passwords.
5. **Traceability:** every file an agent writes records which tool, model and session made it, and which spec version it came from. Every commit starts with the Jira key and the agent's name.

How the Definition of Done is shown, once the first spec exists:

1. **Store:** the spec goes in through a merge request.
2. **Version:** it carries a version number and change log, and is tagged when approved.
3. **Retrieve:** an agent, for example the Testing agent, reads the approved spec. It refuses to use one that isn't approved.

4. **Process:** the agent writes test cases from the spec.
5. **Trace:** the test file's header names the spec version, tool, model and session.

Covers: AC 1 to 14 and the Definition of Done. **Needs:** the GitLab admin for the settings, DevOps for the non-production integration points (AC 11), and the lead for the approvers' names.

5. Step 2: fix the specification standard (Story 1.2, 4 minutes)

Definition of Done: a representative business capability can be fully described without needing the COBOL to explain it.

Show: `agents/templates/program-spec.md`, then `agents/checklists.md`.

Say: "Before the agents write anything, we fix what a good spec looks like. The standard is a set of templates plus one checklist. A program spec has 17 fixed sections: purpose, how it starts, inputs and outputs, data, business rules, flow, state changes, errors, restart and recovery, connections, and more. Module, data and application specs build on it. So we describe the application program by program, then module, then the whole application."

How a business rule is written:

- **Given** the starting situation, **When** the event happens, **Then** the outcome, in business words.
- **Evidence:** the file and lines in iln that prove it.
- **Confidence:** confirmed, high, medium or low. Medium or low must come with an open question for the SME or Product Owner.

Say: "The same checklist is used four times: by the agent before it hands over, by the Verification agent, by Duo Code Review on the merge request, and by the SME and Product Owner when they approve. Every spec carries a version and a change log; the SME marks it reviewed, the Product Owner approves it."

How the Definition of Done is shown: the Product Owner picks one small business capability. The agent writes its program, data and module specs. The Product Owner then explains the capability from the module spec alone, with no COBOL.

Covers: AC 15 to 26 and the Definition of Done. **Needs:** the Product Owner to pick the capability, the SME to review, the lead to approve any change to the templates.

6. Step 3: inventory the iln code (Story 1.3, 4 minutes)

Definition of Done: the pilot source estate is held in an inventory that tools and agents can read.

Show: `agents/templates/inventory.md`, then the inventory part of `agents/specification.md` (step A1).

Say: "The first real work is knowing what iln contains. Without a scanner, we show completeness by counting. The Specification agent goes through the iln folders one at a time, such as cobol, job, procs and ctl, at most 50 members per session."

How it runs:

1. In Bob, mode *iln Specification*: `/inventory-folder cobol, names A to C`. In Duo: "Act as the Specification agent. Do step A1 for folder cobol, members A to C."
2. The agent lists the folder and counts the members.
3. It opens each member and records what it is (program, copybook, job, PROC, CICS or VSAM definition, screen), with the line that proves it.
4. It fills the inventory sections: programs and their calls, copybooks, CICS resources, jobs, VSAM files, datasets, interfaces, existing tests.
5. Anything it can't resolve goes under **Unknowns**, with an owner. It is never presented as fact.
6. The count must add up: members in the folder = members listed + members left out, each with a reason.
7. The index, `specs/iln/index.md`, adds up every part. Its program list becomes the work list for Step 5.

Say: "The result is Markdown with a fixed header and fixed columns, so any tool or agent can read it. That's the machine-readable inventory the story asks for."

Covers: AC 27 to 36 and the Definition of Done. **Needs:** the lead to agree the pilot scope; the SME to review each part; the CICS team, if transaction definitions aren't in the `ctl` folder.

7. Step 4: map the connections and analyse impact (Story 1.4, 4 minutes)

Definition of Done: the graph identifies the components affected by changing a selected program, copybook, VSAM record, CICS transaction or JCL job.

Show: `agents/templates/connections.md`, then `agents/templates/story-impact.md`.

Say: "While inventorying, the agent also records how things connect. The dependency graph is written as tables in the connections spec. Each row says A relates to B: which transaction starts which program, which job runs which step, who calls whom, who reads or writes which file. Every row carries the code line as evidence and a confidence level. GitLab can draw a small diagram from it."

How impact analysis runs:

1. In Bob, mode *iln Jira*: `/impact-analysis copybook <NAME>`. Or ask Duo the same as the Jira agent.
2. The agent starts from the connections spec and follows the links in both directions.
3. It confirms every link with a code search, because specs can go out of date. For example, `COPY <NAME>` finds every program that uses a copybook.
4. It lists what is **affected** (must change or be retested), what is **relied on** (test scope), and what is **possibly affected** (calls through a variable, job order it could only infer) for the SME to confirm.
5. It records the searches it used, so anyone can repeat them.

Say: "The team already does impact analysis in Bob and Duo. This makes it repeatable: the same tables, the searches written down, and evidence for every link."

How the Definition of Done is shown: the SME picks one program, one copybook, one VSAM file, one transaction and one job. The agent runs one impact analysis for each, and the SME checks the results against what they know.

Covers: AC 37 to 46 and the Definition of Done. **Needs:** the SME to pick the five components and review the results.

8. Step 5: extract the business behaviour (Story 1.5, 4 minutes)

Definition of Done: the Product Owner and SME can confirm the extracted behaviour from a readable spec, without reading the COBOL.

Show: the *Code coverage* section of `agents/templates/program-spec.md`, then step A3 in `agents/specification.md`.

Say: "Now the agent specifies each program on the list from Step 3, one program per session."

How it runs:

1. In Bob, mode *iln Specification*: `/specify-program <PROGRAM>`. Or ask Duo to act as the Specification agent for that program.
2. **Retrieve everything the program needs:** its existing specs, the program, its copybooks, the job or transaction that starts it, its files, tables and queues, and the programs it calls. The agent opens every file it will cite.
3. **Read paragraph by paragraph.** Each paragraph is either a business rule, a flow step, a state change, an error path, restart handling, or technical code.

4. **Write the spec:** rules as Given, When, Then, plus the flow, errors, restart and recovery, and data access. Business words, with evidence and a confidence level on everything.
5. **Prove nothing was skipped.** The *Code coverage* section lists every paragraph with its lines and what explains it. The agent also searches the program for decisions, calculations, errors and restart points; every hit must fall in a paragraph it has explained.
6. **Raise open questions** for anything the code doesn't prove.

Say: "When all programs in a business area are done, the agent writes the module spec, the end-to-end flow across those programs. At the end, it writes the application spec."

How the Definition of Done is shown: the Product Owner reads a finished spec in GitLab and explains the behaviour without COBOL; the SME clicks a rule's evidence into the code and confirms it.

Covers: AC 47 to 54 and the Definition of Done. **Needs:** the SME to review each spec, and the Product Owner to answer open questions and approve.

9. Step 6: verify, review and approve (2 minutes)

Every output from Steps 3 to 5 passes the same four checks before it counts.

Say: "Nothing an agent writes is trusted on its own. Each batch goes through four checks."

1. **Verification agent** (`/verify-spec`): goes through the checklist, opens the cited code lines, and repeats a sample of the searches and counts. It reports pass or fail and never fixes anything itself.
2. **Duo Code Review**: comments on the merge request against the same checklist.
3. **Mainframe SME**: confirms the spec matches the code and marks it *reviewed*.
4. **Product Owner**: reads it in business terms, answers the open questions, and approves. The approval is recorded in the spec for that exact version.

Say: "Approved specs are merged to main and tagged as a baseline. Agents may build only on approved specs, never on drafts."

10. Step 7: track progress, and what comes after (1 minute)

Progress is measured in one file, `specs/iln/index.md` : results and use, not lines written.

Show: `agents/templates/spec-index.md` .

Say: "The index shows each folder inventoried, and each program's status: not started, partial, specified, reviewed or approved. A second table records where the specs were used, for example an impact analysis or a question answered. Feature 1 is done when every folder in scope is inventoried and every program is specified and reviewed.

After Feature 1, the same agents deliver Jira stories. The Jira agent does the impact analysis from the specs first, the spec is updated and approved, and only then do the Architect, Coding, Testing and Verification agents run."

Acceptance criteria at a glance

Story	Acceptance criteria	Step	Where they are met
1.1 Environment	AC 1 to 14	1	GitLab, Bob and Duo settings; `AGENTS.md`, agents, modes; tool, model, session and spec version recorded in every output
1.2 Specification standard	AC 15 to 26	2	The templates and the checklist
1.3 Inventory	AC 27 to 36	3	Inventory parts per folder, counted, with unknowns
1.4 Dependency graph	AC 37 to 46	4	The connections spec and impact analyses
1.5 Business behaviour	AC 47 to 54	5	Program and module specs with evidence, confidence and code coverage

Each story's full list of acceptance criteria, with the template section that meets each one, is in `governance/story-1.1-environment.md` to `story-1.5-business-behaviour.md`.

11. Limits, asks and next steps (3 minutes)

Say: "Three limits I want to be open about."

- **Completeness comes from counting and review, not a program.** That's why we work in small batches and the Verification agent and SME repeat some of the checks.
- **It takes many sessions.** Reading iln member by member is slow at its size. I'll time the first module and plan the rest from that, rather than guess now.

- **Tool features vary.** Duo's semantic code search is still beta. The approach works with plain search and reading; extra features only make it faster.

What I need:

Who	What
Lead	Agree the pilot scope; names for the approvers (CODEOWNERS)
GitLab admin	Duo on for `iln-specs` and `iln`; semantic code search if available; context exclusions; Duo Code Review; secret push protection; audit events
DevOps	The non-production integration points (Story 1.1, AC 11)
Product Owner	Pick one capability for Story 1.2; time to read and approve specs
Mainframe SME	A regular slot to review each batch

Next steps:

1. Finish the environment set-up with the admin.
2. Inventory the pilot folders, 50 members per part.
3. Specify the first module's programs, then its module spec.
4. Measure the time taken, plan the rest of iln, and report from the index at the next review.

Say: "To sum up: no scripts, no change to the iln code, and Bob or Duo doing the retrieval and writing, with people approving every result. Thank you. Questions?"

Questions you may be asked

How do we know nothing was missed, without a scanner? Every inventory part's counts must add up against the folder, and every program spec lists every paragraph in *Code coverage*. The Verification agent repeats a sample of the searches and counts, and the SME reviews each batch.

Is this really RAG without our own index? Yes. Retrieval is Bob's and Duo's own search, and Duo's semantic code search where it's on. Our approved specs then become the curated knowledge both tools search first.

What if Bob or Duo makes something up? Every statement must cite lines the agent opened, and the Verification agent and SME open those lines. Anything unproven is marked medium or low confidence, with an open question.

Does any of this change the iln code or start its pipeline? No. The specs live in `iln-specs`. Bob reads `iln` but its modes can't edit it.

How long will the whole of iln take? We'll know after the first module. I'll time it and bring a plan.

Can we use Bob's own mainframe features, such as business-rule extraction? Yes, as a first draft. Its output goes into our template, and every item still needs evidence the agent opened, or it becomes an open question.

What happens when the code changes after a spec is approved? Each spec records the iln commit it was read from. A change comes through a Jira story: impact analysis first, then a spec update, a new version and approval.

Is anything sensitive sent to the model? Only development code. Duo's context exclusions and `.bobignore` keep restricted files out, and no production data is used.

Where is the record of what the agents did? In each file's header (tool, model, session, spec version), the commit messages with the Jira key, the merge requests, and GitLab's audit events.